



**CHENNAI
INSTITUTE OF TECHNOLOGY**
(Autonomous)



(An Autonomous Institution, Affiliated to Anna University, Chennai)

LABORATORY MANUAL

CS4V48 - GENAI AND LLM

(GenAI & LLM Laboratory)

INDEX

Ex. No.	Name of the Experiment
1	Text Generation Using Pre-Trained Foundation Models
2	Prompt Engineering Techniques For Content Generation, Reasoning And Task Automation
3	Conversational AI Chatbot Using Transformer-Based Language Models
4	Text Summarization And Question-Answering System Using Large Language Models
5	Sentiment Analysis And Document Classification Using Foundation Models
6	Retrieval-Augmented Generation (RAG) System Using Vector Databases
7	AI-Powered Code Generation And Debugging Assistant
8	Image Generation Application Using Diffusion Models
9	Multimodal AI Application Integrating Text And Image Inputs
10	Fine-Tuning A Pre-Trained Language Model For A Domain-Specific Application
11	AI-Based Content Generation System For Text, Image And Multimedia Applications
12	Deployment And Evaluation Of A Generative AI Application Using Cloud-Based APIs And AI Frameworks

Ex. No: 1**TEXT GENERATION USING PRE-TRAINED FOUNDATION MODELS**

AIM

To develop a text generation application using a pre-trained foundation model (GPT-2) with the Hugging Face Transformers library.

OBJECTIVE

To understand how pre-trained foundation models generate coherent text from a given prompt, and to explore decoding strategies such as greedy search, sampling, and top-k / top-p (nucleus) sampling that control the quality and diversity of generated text.

SOFTWARE REQUIREMENTS

- Python 3.9 or above
- Hugging Face transformers library (pip install transformers)
- PyTorch (pip install torch)
- Google Colab / Jupyter Notebook / VS Code

THEORY

A foundation model is a large neural network pre-trained on massive amounts of text data in a self-supervised manner, which can then be adapted to a wide range of downstream tasks. GPT-2 (Generative Pre-trained Transformer 2) is a decoder-only transformer model trained to predict the next token in a sequence given all previous tokens.

Text generation works by feeding a prompt (seed text) into the model, which computes a probability distribution over the vocabulary for the next token. The next token is chosen using a decoding strategy — greedy decoding always picks the highest probability token, while sampling based methods (temperature, top-k, top-p) introduce controlled randomness to make the generated text more natural and diverse.

The Hugging Face 'pipeline' API abstracts away tokenization, model loading, and decoding, allowing text generation to be performed with just a few lines of code.

ALGORITHM

1. Install and import the transformers library.
2. Load a pre-trained text-generation pipeline with a chosen foundation model (e.g., GPT-2).
3. Provide an input prompt string.
4. Set generation parameters — max_length, num_return_sequences, temperature, top_k, top_p.
5. Call the pipeline on the prompt to generate text.
6. Display and analyse the generated output.

PROGRAM / CODE

```
from transformers import pipeline, set_seed

# Load the pre-trained GPT-2 text generation pipeline
generator = pipeline("text-generation", model="gpt2")
set_seed(42)

# Input prompt
```

```
prompt = "Artificial Intelligence will transform the future of"
```

```
# Generate text
outputs = generator(
    prompt,
    max_length=60,
    num_return_sequences=2,
    temperature=0.8,
    top_k=50,
    top_p=0.95,
    do_sample=True
)

for i, out in enumerate(outputs, 1):
    print(f"--- Generated Text {i} ---")
    print(out["generated_text"])
    print()
```

SAMPLE INPUT

```
prompt = "Artificial Intelligence will transform the future of"
```

SAMPLE OUTPUT

```
--- Generated Text 1 ---
Artificial Intelligence will transform the future of healthcare, education,
and transportation by enabling smarter decision making and automating
repetitive tasks across industries.

--- Generated Text 2 ---
Artificial Intelligence will transform the future of work by creating new
job roles while automating routine processes in manufacturing and services.
```

RESULT

A text generation application using the pre-trained GPT-2 foundation model was successfully developed, and coherent text was generated from a given prompt using sampling-based decoding strategies.

Ex. No: 2

PROMPT ENGINEERING TECHNIQUES FOR CONTENT GENERATION, REASONING AND TASK AUTOMATION

AIM

To implement zero-shot, one-shot, few-shot, and chain-of-thought prompting techniques for content generation, reasoning, and task automation using a Large Language Model.

OBJECTIVE

To understand how the structure and content of a prompt influences the output quality of an LLM, and to practically apply different prompting strategies to solve content generation and reasoning tasks without modifying model weights.

SOFTWARE REQUIREMENTS

- Python 3.9 or above
- OpenAI Python SDK / Hugging Face transformers
- Valid API key (if using OpenAI/Gemini) or a local pipeline model
- Jupyter Notebook / VS Code

THEORY

Prompt engineering is the practice of designing the input text (prompt) given to a Large Language Model so as to elicit the most accurate and useful response, without changing the model's parameters. It leverages the model's in-context learning ability.

Zero-shot prompting asks the model to perform a task with no examples, relying purely on instructions. One-shot and few-shot prompting provide one or more example input-output pairs within the prompt so the model can infer the expected pattern. Chain-of-Thought (CoT) prompting encourages the model to reason step-by-step before producing a final answer, which significantly improves performance on arithmetic, logical, and multi-step reasoning tasks.

Choosing the right prompting technique is crucial for tasks like content generation, reasoning, classification, and automating repetitive text-based workflows.

ALGORITHM

1. Define the task to be solved (e.g., arithmetic reasoning, content generation, classification).
2. Construct a zero-shot prompt containing only the instruction and query.
3. Construct a few-shot prompt containing 2–3 example input-output pairs followed by the query.
4. Construct a chain-of-thought prompt that asks the model to 'think step by step'.
5. Send each prompt to the LLM and record the response.
6. Compare the outputs of the three techniques for correctness and quality.

PROGRAM / CODE

```
from transformers import pipeline

generator = pipeline("text-generation", model="gpt2")

# 1. Zero-shot prompt
```

```
zero_shot_prompt = "Classify the sentiment of this review as Positive or Negative: 'The product quality is excellent!'\nSentiment:"
```

```
# 2. Few-shot prompt
```

```
few_shot_prompt = ""Review: 'I loved this movie, it was fantastic.'
```

```
Sentiment: Positive
```

```
Review: 'The service was slow and disappointing.'
```

```
Sentiment: Negative
```

```
Review: 'The product quality is excellent!'
```

```
Sentiment: ""
```

```
# 3. Chain-of-Thought prompt
```

```
cot_prompt = ""Q: A shop had 15 apples. It sold 6 and then received 10 more. How many apples now?
```

```
A: Let's think step by step. 15 - 6 = 9. 9 + 10 = 19. The answer is 19.
```

```
Q: A library had 120 books. It lent out 45 and bought 30 new books. How many books now?
```

```
A: Let's think step by step.
```

```
for name, p in [("Zero-shot", zero_shot_prompt),
```

```
                ("Few-shot", few_shot_prompt),
```

```
                ("Chain-of-Thought", cot_prompt)]:
```

```
    out = generator(p, max_length=len(p.split())+40, num_return_sequences=1, do_sample=False)
```

```
    print(f"=== {name} ===")
```

```
    print(out[0]["generated_text"])
```

```
    print()
```

SAMPLE INPUT

Sentiment classification query and a two-step arithmetic word problem (see prompts above)

SAMPLE OUTPUT

```
=== Zero-shot ===
```

```
Sentiment: Positive
```

```
=== Few-shot ===
```

```
Review: 'The product quality is excellent!'
```

```
Sentiment: Positive
```

```
=== Chain-of-Thought ===
```

```
A: Let's think step by step. 120 - 45 = 75. 75 + 30 = 105. The answer is 105.
```

RESULT

Zero-shot, few-shot, and chain-of-thought prompting techniques were successfully implemented, and their impact on content generation and reasoning accuracy was analysed and compared.

Ex. No: 3

CONVERSATIONAL AI CHATBOT USING TRANSFORMER-BASED LANGUAGE MODELS

AIM

To build a conversational AI chatbot capable of holding a multi-turn dialogue using a transformer-based language model (DialoGPT).

OBJECTIVE

To understand how transformer decoder models maintain conversational context across multiple turns using dialogue history encoding, and to implement a simple interactive chatbot.

SOFTWARE REQUIREMENTS

- Python 3.9 or above
- Hugging Face transformers and torch libraries
- Pre-trained model: microsoft/DialoGPT-medium
- Jupyter Notebook / VS Code

THEORY

Conversational AI chatbots use transformer-based language models trained on large volumes of dialogue data to generate human-like responses. DialoGPT is a GPT-2-based model fine-tuned on Reddit conversation threads, specifically designed for open-domain, multi-turn dialogue generation.

To maintain context, the chatbot concatenates the current user input with the previous conversation turns (encoded as token ids) before generating a response, so that the model 'remembers' what was said earlier in the session. The response is generated using sampling or beam search and decoded back into readable text.

Such chatbots form the basis of virtual assistants, customer-support bots, and other interactive natural-language interfaces.

ALGORITHM

1. Load the DialoGPT tokenizer and model.
2. Initialise an empty chat-history tensor.
3. In a loop, accept user input and encode it, appending an end-of-sentence token.
4. Concatenate the new input with the existing chat history.
5. Generate a response from the model using the combined history as context.
6. Decode and display only the newly generated tokens as the bot's reply.
7. Update the chat history and repeat for the next turn.

PROGRAM / CODE

```
from transformers import AutoModelForCausalLM, AutoTokenizer
import torch

tokenizer = AutoTokenizer.from_pretrained("microsoft/DialoGPT-medium")
model = AutoModelForCausalLM.from_pretrained("microsoft/DialoGPT-medium")

chat_history_ids = None
```

```

print("Chatbot ready! Type 'quit' to exit.")
for step in range(5):
    user_input = input(">> User: ")
    if user_input.lower() == "quit":
        break

    new_input_ids = tokenizer.encode(user_input + tokenizer.eos_token, return_tensors="pt")
    bot_input_ids = torch.cat([chat_history_ids, new_input_ids], dim=-1) if chat_history_ids is not None else new_input_ids

    chat_history_ids = model.generate(
        bot_input_ids,
        max_length=1000,
        pad_token_id=tokenizer.eos_token_id,
        do_sample=True,
        top_k=50,
        top_p=0.9
    )

    response = tokenizer.decode(chat_history_ids[:, bot_input_ids.shape[-1]:][0], skip_special_tokens=True)
    print(f"Bot: {response}")

```

SAMPLE INPUT

>> User: Hi, how are you? >> User: What can you help me with?

SAMPLE OUTPUT

Bot: I'm doing great, thanks for asking! How about you?
 Bot: I can chat with you about almost anything - just ask away!

RESULT

A multi-turn conversational chatbot was successfully built using the transformer-based DialoGPT model, capable of retaining dialogue context across turns.

Ex. No: 4

TEXT SUMMARIZATION AND QUESTION-ANSWERING SYSTEM USING LARGE LANGUAGE MODELS

AIM

To develop a text summarization system and a question-answering system using pre-trained Large Language Models (BART and DistilBERT).

OBJECTIVE

To understand abstractive summarization and extractive question-answering pipelines built on transformer encoder-decoder and encoder-only architectures respectively.

SOFTWARE REQUIREMENTS

- Python 3.9 or above
- Hugging Face transformers library
- Pre-trained models: facebook/bart-large-cnn (summarization), distilbert-base-cased-distilled-squad (QA)
- Jupyter Notebook / VS Code

THEORY

Text summarization condenses a long passage into a shorter version while preserving its key information. Abstractive summarization models such as BART use an encoder-decoder transformer architecture: the encoder reads the full document and the decoder generates a new, fluent summary rather than merely extracting sentences.

Question-Answering (QA) systems extract or generate an answer to a natural-language question from a given context passage. Extractive QA models such as DistilBERT (fine-tuned on the SQuAD dataset) predict the start and end token positions of the answer span within the context passage using a classification head placed over the encoder outputs.

Both tasks are commonly exposed through the Hugging Face 'pipeline' API for rapid application development.

ALGORITHM

1. Load the summarization pipeline with a pre-trained BART model.
2. Provide a long passage of text and generate a summary with defined min/max length.
3. Load the question-answering pipeline with a pre-trained DistilBERT-SQuAD model.
4. Provide a context passage and a natural-language question.
5. Run the QA pipeline to extract the answer span along with a confidence score.
6. Display the summary and the answer.

PROGRAM / CODE

```
from transformers import pipeline

# ----- Text Summarization -----
summarizer = pipeline("summarization", model="facebook/bart-large-cnn")

article = """Generative AI refers to a class of artificial intelligence models capable of producing new content such as text, images, audio, and video. Large Language Models (LLMs) such as GPT and LLaMA are trained on massive text corpora and can perform a wide range of natural language tasks including translation, summarization, and question answering. These
```

models are increasingly being deployed in industry applications ranging from customer support to software development, transforming how humans interact with machines.'''

```
summary = summarizer(article, max_length=45, min_length=20, do_sample=False)
print("Summary:\n", summary[0]["summary_text"])
```

```
# ----- Question Answering -----
```

```
qa = pipeline("question-answering", model="distilbert-base-cased-distilled-squad")
```

```
context = article
```

```
question = "What are Large Language Models trained on?"
```

```
answer = qa(question=question, context=context)
```

```
print("\nQuestion:", question)
```

```
print("Answer:", answer["answer"], "| Confidence:", round(answer["score"], 3))
```

SAMPLE INPUT

Article about Generative AI and LLMs (see code) + Question: 'What are Large Language Models trained on?'

SAMPLE OUTPUT

Summary:

Generative AI models produce new content such as text, images, audio and video.

Large Language Models are trained on massive text corpora and perform many NLP tasks.

Question: What are Large Language Models trained on?

Answer: massive text corpora | Confidence: 0.87

RESULT

A text summarization system using BART and a question-answering system using DistilBERT-SQuAD were successfully developed and tested on sample text.

Ex. No: 5

SENTIMENT ANALYSIS AND DOCUMENT CLASSIFICATION USING FOUNDATION MODELS

AIM

To perform sentiment analysis and multi-class document classification using pre-trained foundation models.

OBJECTIVE

To understand how foundation models fine-tuned for classification tasks, and zero-shot classification models based on natural language inference, can be applied to categorise text without task-specific training.

SOFTWARE REQUIREMENTS

- Python 3.9 or above
- Hugging Face transformers library
- Pre-trained models: distilbert-base-uncased-finetuned-sst-2-english (sentiment), facebook/bart-large-mnli (zero-shot classification)
- Jupyter Notebook / VS Code

THEORY

Sentiment analysis classifies text according to the emotional tone expressed (e.g., Positive/Negative). Foundation models fine-tuned on labelled sentiment datasets such as SST-2 attach a classification head on top of a pre-trained transformer encoder to predict sentiment labels with associated confidence scores.

Document classification assigns a document to one or more predefined categories. Zero-shot classification models (e.g., BART-large-MNLI) reformulate classification as a Natural Language Inference (NLI) problem — the document is treated as a 'premise' and each candidate label is turned into a 'hypothesis' (e.g., 'This text is about {label}'); the model outputs the entailment probability for every label, enabling classification into arbitrary, previously unseen categories without any additional training.

This approach demonstrates the versatility of foundation models for text-categorisation tasks.

ALGORITHM

1. Load the sentiment-analysis pipeline with a fine-tuned foundation model.
2. Pass sample review sentences to the pipeline and record sentiment + score.
3. Load the zero-shot-classification pipeline with BART-large-MNLI.
4. Define a document and a list of candidate category labels.
5. Run zero-shot classification to obtain the probability for each label.
6. Display sentiment results and the top predicted document category.

PROGRAM / CODE

```
from transformers import pipeline

# ----- Sentiment Analysis -----
sentiment_analyzer = pipeline("sentiment-analysis")

reviews = [
    "The new smartphone has an amazing camera and battery life!",
    "The delivery was late and the packaging was damaged."
```

```

]
for review in reviews:
    result = sentiment_analyzer(review)[0]
    print(f"Review: {review}\n -> {result['label']} ( {round(result['score'],3)})\n")

# ----- Document Classification (Zero-Shot) -----
classifier = pipeline("zero-shot-classification", model="facebook/bart-large-mnli")

document = "The central bank raised interest rates to control rising inflation."
candidate_labels = ["Politics", "Economy", "Sports", "Technology"]

classification = classifier(document, candidate_labels)
print("Document:", document)
for label, score in zip(classification["labels"], classification["scores"]):
    print(f"{label}: {round(score,3)}")

```

SAMPLE INPUT

Two product reviews (sentiment) + one news sentence with candidate labels [Politics, Economy, Sports, Technology]

SAMPLE OUTPUT

```

Review: The new smartphone has an amazing camera and battery life!
-> POSITIVE (0.999)

Review: The delivery was late and the packaging was damaged.
-> NEGATIVE (0.998)

Document: The central bank raised interest rates to control rising inflation.
Economy: 0.94
Politics: 0.04
Technology: 0.01
Sports: 0.01

```

RESULT

Sentiment analysis and zero-shot document classification were successfully performed using pre-trained foundation models, correctly identifying sentiment polarity and document category.

Ex. No: 6

RETRIEVAL-AUGMENTED GENERATION (RAG) SYSTEM USING VECTOR DATABASES

AIM

To build a Retrieval-Augmented Generation (RAG) system that retrieves relevant document chunks from a vector database and uses an LLM to generate grounded answers.

OBJECTIVE

To understand how embeddings, vector similarity search, and LLM generation are combined to reduce hallucination and answer questions using an external knowledge base.

SOFTWARE REQUIREMENTS

- Python 3.9 or above
- langchain, sentence-transformers, faiss-cpu libraries
- Hugging Face transformers (for the generator LLM)
- Jupyter Notebook / VS Code

THEORY

Retrieval-Augmented Generation (RAG) combines a retriever and a generator to answer queries using knowledge that may not be present in the LLM's parameters. Documents are split into chunks, converted into dense vector embeddings using a sentence-embedding model, and stored in a vector database (e.g., FAISS, Chroma, Pinecone).

At query time, the question is also embedded, and a similarity search (commonly cosine similarity or L2 distance) retrieves the top-k most relevant chunks. These retrieved chunks are inserted into the LLM's prompt as context, and the LLM generates an answer grounded in the retrieved evidence.

RAG mitigates hallucination, allows knowledge updates without retraining the model, and enables domain-specific question answering over private document collections.

ALGORITHM

1. Collect and split source documents into smaller text chunks.
2. Generate vector embeddings for each chunk using a sentence-transformer model.
3. Store the embeddings in a FAISS vector index.
4. Embed the incoming user query using the same embedding model.
5. Perform similarity search to retrieve the top-k most relevant chunks.
6. Construct a prompt combining the retrieved chunks and the query.
7. Pass the prompt to an LLM to generate the final grounded answer.

PROGRAM / CODE

```
from sentence_transformers import SentenceTransformer
import faiss
import numpy as np
from transformers import pipeline

# 1. Knowledge base
documents = [
    "The Eiffel Tower is located in Paris, France and was completed in 1889.",
```

```

"Retrieval-Augmented Generation combines document retrieval with text generation.",
"Python is a popular high-level programming language used in AI development.",
"Vector databases store embeddings and support fast similarity search."
]

# 2. Embed documents
embed_model = SentenceTransformer("all-MiniLM-L6-v2")
doc_embeddings = embed_model.encode(documents)

# 3. Build FAISS index
dimension = doc_embeddings.shape[1]
index = faiss.IndexFlatL2(dimension)
index.add(np.array(doc_embeddings))

# 4. Query and retrieve top-2 relevant chunks
query = "What is RAG in AI?"
query_embedding = embed_model.encode([query])
D, I = index.search(np.array(query_embedding), k=2)
retrieved_chunks = [documents[i] for i in I[0]]

# 5. Build augmented prompt and generate answer
context = " ".join(retrieved_chunks)
prompt = f"Context: {context}\nQuestion: {query}\nAnswer:"

generator = pipeline("text2text-generation", model="google/flan-t5-base")
answer = generator(prompt, max_length=60)
print("Retrieved Context:", retrieved_chunks)
print("Answer:", answer[0]["generated_text"])

```

SAMPLE INPUT

```
query = "What is RAG in AI?"
```

SAMPLE OUTPUT

```

Retrieved Context: ['Retrieval-Augmented Generation combines document retrieval with
text generation.', 'Vector databases store embeddings and support fast similarity search.']
Answer: RAG combines document retrieval with text generation using vector databases.

```

RESULT

A Retrieval-Augmented Generation system was successfully implemented using FAISS as the vector database, producing answers grounded in retrieved document context.

Ex. No: 7

AI-POWERED CODE GENERATION AND DEBUGGING ASSISTANT

AIM

To develop an AI-powered assistant that generates code from natural-language descriptions and helps identify/fix bugs in existing code using a pre-trained code-generation model.

OBJECTIVE

To understand how transformer models trained on source-code corpora (code LLMs) can automate code generation, code explanation, and debugging tasks.

SOFTWARE REQUIREMENTS

- Python 3.9 or above
- Hugging Face transformers library
- Pre-trained model: Salesforce/codegen-350M-mono (or OpenAI Codex/GPT API)
- Jupyter Notebook / VS Code

THEORY

Code-generation LLMs (e.g., CodeGen, StarCoder, Codex) are transformer decoder models pre-trained on large corpora of source code and natural-language comments. Given a natural-language instruction or a partial code snippet as prompt, the model predicts the most likely continuation, effectively 'writing' code.

For debugging, the buggy code along with an instruction (e.g., 'Fix the bug in this function') is passed as the prompt; the model uses patterns learned from millions of code examples to suggest corrected code or explain the likely cause of the error.

Such AI coding assistants (similar in spirit to GitHub Copilot) significantly speed up software development by automating boilerplate code writing and initial debugging.

ALGORITHM

1. Load a pre-trained code-generation model and its tokenizer.
2. Provide a natural-language instruction describing the required function.
3. Generate code using the model and display the output.
4. Provide a snippet of buggy code with an explicit 'fix the bug' instruction.
5. Generate the corrected code using the model.
6. Compare the buggy and fixed versions to verify correctness.

PROGRAM / CODE

```
from transformers import AutoTokenizer, AutoModelForCausalLM
import torch

tokenizer = AutoTokenizer.from_pretrained("Salesforce/codegen-350M-mono")
model = AutoModelForCausalLM.from_pretrained("Salesforce/codegen-350M-mono")

def generate_code(prompt, max_new_tokens=80):
    input_ids = tokenizer(prompt, return_tensors="pt").input_ids
    output = model.generate(input_ids, max_new_tokens=max_new_tokens,
                           pad_token_id=tokenizer.eos_token_id, do_sample=False)
    return tokenizer.decode(output[0], skip_special_tokens=True)
```

```
# 1. Code generation from a natural-language instruction
prompt1 = "# Write a Python function to check if a number is prime\ndef is_prime(n):"
print("Generated Function:\n", generate_code(prompt1))

# 2. Debugging a faulty snippet
buggy_code = """"# The following function should return the factorial of n, but has a bug. Fix it.
def factorial(n):
    result = 0
    for i in range(1, n+1):
        result = result * i
    return result
# Corrected function:
def factorial_fixed(n):""""
print("\nDebug Suggestion:\n", generate_code(buggy_code, max_new_tokens=60))
```

SAMPLE INPUT

Instruction: 'check if a number is prime' + buggy factorial() function initialised with result = 0

SAMPLE OUTPUT

```
Generated Function:
def is_prime(n):
    if n < 2:
        return False
    for i in range(2, int(n ** 0.5) + 1):
        if n % i == 0:
            return False
    return True

Debug Suggestion:
def factorial_fixed(n):
    result = 1
    for i in range(1, n+1):
        result = result * i
    return result
```

RESULT

An AI-powered assistant was successfully developed that generates correct Python code from natural-language instructions and identifies/fixes bugs in existing code using a pre-trained code LLM.

Ex. No: 8

IMAGE GENERATION APPLICATION USING DIFFUSION MODELS

AIM

To implement an image generation application using a pre-trained Diffusion Model (Stable Diffusion) that synthesises images from text prompts.

OBJECTIVE

To understand the working principle of diffusion-based generative models — the forward noising process and the reverse denoising process — and to use them for text-to-image generation.

SOFTWARE REQUIREMENTS

- Python 3.9 or above
- diffusers, transformers, torch, accelerate libraries
- Pre-trained model: runwayml/stable-diffusion-v1-5
- GPU-enabled environment (Google Colab with GPU runtime recommended)

THEORY

Diffusion models are generative models that learn to reverse a gradual noising process. During training, Gaussian noise is progressively added to real images over many timesteps (forward process) until the image becomes pure noise. The model — typically a U-Net — is trained to predict and remove this noise at each timestep (reverse process).

At inference time, the model starts from pure random noise and iteratively denoises it, step by step, guided by a text prompt encoded via a text encoder (e.g., CLIP), gradually 'sculpting' a coherent image that matches the prompt's description. This is the mechanism behind text-to-image models such as Stable Diffusion, DALL-E 2, and Imagen.

Diffusion models currently produce state-of-the-art image quality and diversity, outperforming earlier GAN-based approaches in many benchmarks, though they can be compared against GAN architectures for speed versus quality trade-offs.

ALGORITHM

1. Load the pre-trained Stable Diffusion pipeline onto the GPU.
2. Define a descriptive text prompt for the desired image.
3. Set the number of inference (denoising) steps and guidance scale.
4. Run the pipeline: start from random noise and iteratively denoise conditioned on the prompt.
5. Retrieve and save/display the final generated image.
6. Experiment with different prompts and guidance-scale values to observe quality changes.

PROGRAM / CODE

```
from diffusers import StableDiffusionPipeline
import torch

pipe = StableDiffusionPipeline.from_pretrained(
    "runwayml/stable-diffusion-v1-5",
    torch_dtype=torch.float16
)
pipe = pipe.to("cuda")
```

```
prompt = "A futuristic city skyline at sunset, digital art, highly detailed"
```

```
image = pipe(  
    prompt,  
    num_inference_steps=30,  
    guidance_scale=7.5  
)  
.images[0]
```

```
image.save("generated_city.png")  
print("Image generated and saved as generated_city.png")
```

SAMPLE INPUT

prompt = "A futuristic city skyline at sunset, digital art, highly detailed"

SAMPLE OUTPUT

Image generated and saved as generated_city.png
(A 512x512 PNG image showing a futuristic city skyline with warm sunset lighting is produced and saved to disk.)

RESULT

An image generation application using the Stable Diffusion diffusion model was successfully implemented, generating a realistic image from a given text prompt.

Ex. No: 9

MULTIMODAL AI APPLICATION INTEGRATING TEXT AND IMAGE INPUTS

AIM

To develop a multimodal AI application that processes both text and image inputs using a pre-trained vision-language model (BLIP) to perform image captioning and visual question answering.

OBJECTIVE

To understand how multimodal foundation models jointly encode visual and textual information into a shared representation space to perform cross-modal tasks.

SOFTWARE REQUIREMENTS

- Python 3.9 or above
- Hugging Face transformers, Pillow libraries
- Pre-trained model: Salesforce/blip-image-captioning-base, Salesforce/blip-vqa-base
- Jupyter Notebook / VS Code

THEORY

Multimodal AI systems combine information from more than one modality — such as text and image — to perform tasks that neither modality could solve alone. Vision-language models like BLIP (Bootstrapped Language-Image Pre-training) use a vision encoder (typically a Vision Transformer) to convert an image into a sequence of visual features, and a text encoder/decoder to process language.

These visual and textual representations are fused (often via cross-attention layers) into a joint embedding space, enabling tasks such as image captioning (generating a text description of an image) and Visual Question Answering — VQA (answering a natural-language question about the content of an image).

Multimodal models power applications such as accessibility tools (image descriptions for the visually impaired), content moderation, and interactive visual assistants.

ALGORITHM

1. Load a pre-trained BLIP model and processor for image captioning.
2. Load an input image using PIL.
3. Pass the image through the processor and model to generate a caption.
4. Load a BLIP-VQA model and processor.
5. Provide the same image along with a natural-language question.
6. Generate and display the answer produced by the model.

PROGRAM / CODE

```
from transformers import BlipProcessor, BlipForConditionalGeneration, BlipForQuestionAnswering
from PIL import Image
import requests

image_url = "https://images.unsplash.com/photo-1519125323398-675f0ddb6308"
raw_image = Image.open(requests.get(image_url, stream=True).raw).convert("RGB")

# ----- Image Captioning -----
```

```
cap_processor = BlipProcessor.from_pretrained("Salesforce/blip-image-captioning-base")
cap_model = BlipForConditionalGeneration.from_pretrained("Salesforce/blip-image-captioning-base")

inputs = cap_processor(raw_image, return_tensors="pt")
caption_ids = cap_model.generate(**inputs, max_new_tokens=30)
caption = cap_processor.decode(caption_ids[0], skip_special_tokens=True)
print("Generated Caption:", caption)

# ----- Visual Question Answering -----
vqa_processor = BlipProcessor.from_pretrained("Salesforce/blip-vqa-base")
vqa_model = BlipForQuestionAnswering.from_pretrained("Salesforce/blip-vqa-base")

question = "What animal is in the picture?"
vqa_inputs = vqa_processor(raw_image, question, return_tensors="pt")
answer_ids = vqa_model.generate(**vqa_inputs)
answer = vqa_processor.decode(answer_ids[0], skip_special_tokens=True)
print("Question:", question)
print("Answer:", answer)
```

SAMPLE INPUT

Image of a dog in a field (URL) + Question: 'What animal is in the picture?'

SAMPLE OUTPUT

```
Generated Caption: a dog running through a grassy field
Question: What animal is in the picture?
Answer: dog
```

RESULT

A multimodal AI application integrating text and image inputs was successfully developed using the BLIP model, performing both image captioning and visual question answering.

Ex. No: 10

FINE-TUNING A PRE-TRAINED LANGUAGE MODEL FOR A DOMAIN-SPECIFIC APPLICATION

AIM

To fine-tune a pre-trained language model (DistilBERT) on a domain-specific labelled dataset for a text-classification task.

OBJECTIVE

To understand the process of transfer learning — adapting a general-purpose pre-trained model to a specific downstream task and domain by continuing training on a smaller, labelled dataset.

SOFTWARE REQUIREMENTS

- Python 3.9 or above
- transformers, datasets, torch, scikit-learn libraries
- Pre-trained model: distilbert-base-uncased
- GPU-enabled environment recommended (Google Colab)

THEORY

Fine-tuning adapts a pre-trained foundation model to a specific downstream task by continuing training on a smaller, task/domain-specific labelled dataset, updating some or all of the model's weights. This leverages the general linguistic knowledge already captured during pre-training, requiring far less data and compute than training from scratch.

For text classification, a classification head (a linear layer) is added on top of the pre-trained transformer encoder's pooled output. During fine-tuning, cross-entropy loss between the predicted class probabilities and the true labels is minimised using gradient descent (typically with the AdamW optimizer and a small learning rate) over a few epochs.

Fine-tuning is widely used to specialise LLMs for domains such as medical, legal, or customer-support text classification, sentiment detection, and intent recognition.

ALGORITHM

1. Load a domain-specific labelled dataset (text + label pairs).
2. Load the pre-trained tokenizer and tokenize the dataset (padding/truncation).
3. Load the pre-trained model with a classification head matching the number of labels.
4. Define training arguments (learning rate, epochs, batch size).
5. Train the model on the training split using the Hugging Face Trainer API.
6. Evaluate the fine-tuned model on the validation/test split.
7. Save the fine-tuned model for later inference.

PROGRAM / CODE

```
from datasets import load_dataset
from transformers import (AutoTokenizer, AutoModelForSequenceClassification,
                          TrainingArguments, Trainer)
import numpy as np
from sklearn.metrics import accuracy_score
```

```

# 1. Load a domain-specific dataset (example: IMDB movie reviews)
dataset = load_dataset("imdb")
small_train = dataset["train"].shuffle(seed=42).select(range(2000))
small_test = dataset["test"].shuffle(seed=42).select(range(500))

# 2. Tokenize
tokenizer = AutoTokenizer.from_pretrained("distilbert-base-uncased")
def tokenize(batch):
    return tokenizer(batch["text"], padding="max_length", truncation=True, max_length=128)

train_ds = small_train.map(tokenize, batched=True)
test_ds = small_test.map(tokenize, batched=True)

# 3. Load pre-trained model with classification head
model = AutoModelForSequenceClassification.from_pretrained(
    "distilbert-base-uncased", num_labels=2
)

# 4. Training arguments
args = TrainingArguments(
    output_dir="./results",
    num_train_epochs=2,
    per_device_train_batch_size=8,
    per_device_eval_batch_size=8,
    eval_strategy="epoch",
    logging_steps=50
)

def compute_metrics(pred):
    preds = np.argmax(pred.predictions, axis=1)
    return {"accuracy": accuracy_score(pred.label_ids, preds)}

# 5. Train
trainer = Trainer(model=model, args=args, train_dataset=train_ds,
                  eval_dataset=test_ds, compute_metrics=compute_metrics)
trainer.train()

# 6. Evaluate and save
metrics = trainer.evaluate()
print("Evaluation metrics:", metrics)
model.save_pretrained("./fine_tuned_distilbert_imdb")

```

SAMPLE INPUT

IMDB movie review dataset (2000 training samples, 500 test samples), 2 epochs fine-tuning

SAMPLE OUTPUT

```

Epoch 1/2 - loss: 0.41 - accuracy: 0.83
Epoch 2/2 - loss: 0.24 - accuracy: 0.89
Evaluation metrics: {'eval_loss': 0.28, 'eval_accuracy': 0.887}

```

RESULT

The pre-trained DistilBERT model was successfully fine-tuned on a domain-specific sentiment-classification dataset, improving task-specific accuracy through transfer learning.

Ex. No: 11

AI-BASED CONTENT GENERATION SYSTEM FOR TEXT, IMAGE AND MULTIMEDIA APPLICATIONS

AIM

To develop an integrated AI-based content generation system that produces text, image and audio content from a single high-level user input.

OBJECTIVE

To understand how multiple generative models — an LLM for text, a diffusion model for images, and a text-to-speech model for audio — can be orchestrated together to build an end-to-end multimedia content-generation pipeline.

SOFTWARE REQUIREMENTS

- Python 3.9 or above
- transformers, diffusers, gTTS (or a TTS model) libraries
- Pre-trained models: gpt2 / flan-t5 (text), stable-diffusion-v1-5 (image), gTTS (audio)
- Jupyter Notebook / VS Code, GPU recommended for image generation

THEORY

AI-based content generation systems automate the creation of blog posts, social-media captions, marketing images, and narrated audio by chaining together multiple generative models, each specialised for one modality. A pipeline approach is typically used: the LLM first generates textual content (e.g., a short article or caption) from a topic; that generated text (or a derived prompt) is then passed to an image-generation model to create matching visuals; finally, a text-to-speech (TTS) model converts the generated text into narrated audio.

Such orchestration requires careful prompt design at each stage so that the output of one model becomes a coherent input for the next, and demonstrates how foundation models across modalities can be composed into practical multimedia-content workflows used in marketing, education, and social media automation.

ALGORITHM

1. Accept a high-level topic/theme from the user.
2. Use a text-generation LLM to produce a short article/caption on the topic.
3. Derive an image prompt from the generated text.
4. Use a diffusion model to generate an accompanying image from the image prompt.
5. Convert the generated text into speech using a text-to-speech model.
6. Save/display the generated text, image, and audio as the final multimedia content package.

PROGRAM / CODE

```
from transformers import pipeline
from diffusers import StableDiffusionPipeline
from gtts import gTTS
import torch

topic = "The benefits of renewable energy"

# 1. Text generation
```



```
text_generator = pipeline("text2text-generation", model="google/flan-t5-base")
text_prompt = f"Write a short, engaging paragraph about: {topic}"
generated_text = text_generator(text_prompt, max_length=80)[0]["generated_text"]
print("Generated Text:\n", generated_text)

# 2. Image generation (derived prompt)
image_prompt = f"An illustration representing {topic}, digital art"
sd_pipe = StableDiffusionPipeline.from_pretrained(
    "runwayml/stable-diffusion-v1-5", torch_dtype=torch.float16).to("cuda")
image = sd_pipe(image_prompt, num_inference_steps=25).images[0]
image.save("content_image.png")
print("Image saved as content_image.png")

# 3. Audio generation (text-to-speech)
tts = gTTS(text=generated_text, lang="en")
tts.save("content_audio.mp3")
print("Audio saved as content_audio.mp3")
```

SAMPLE INPUT

topic = "The benefits of renewable energy"

SAMPLE OUTPUT

Generated Text:
Renewable energy sources like solar and wind reduce carbon emissions,
lower energy costs over time, and help create a sustainable future for
generations to come.
Image saved as content_image.png
Audio saved as content_audio.mp3

RESULT

An integrated AI-based content generation system producing text, image, and audio outputs from a single topic input was successfully developed.

Ex. No: 12

DEPLOYMENT AND EVALUATION OF A GENERATIVE AI APPLICATION USING CLOUD-BASED APIs AND AI FRAMEWORKS

AIM

To deploy a Generative AI text-generation application as a web service using the Gradio framework, and to evaluate its output quality using standard NLP evaluation metrics.

OBJECTIVE

To understand the end-to-end lifecycle of a Generative AI application — from wrapping a model into a user-facing interface, to deploying it as a shareable web app, and quantitatively evaluating the quality of its generated output.

SOFTWARE REQUIREMENTS

- Python 3.9 or above
- gradio, transformers, evaluate, rouge_score libraries
- OpenAI / Hugging Face Inference API (optional, for a cloud-hosted model)
- Jupyter Notebook / VS Code / Hugging Face Spaces for hosting

THEORY

Deploying a Generative AI application involves wrapping a trained/pre-trained model behind a user interface or API endpoint so that end-users can interact with it without needing to run code themselves. Frameworks such as Gradio and Streamlit allow a model inference function to be exposed as an interactive web application with just a few lines of code, and can be hosted on cloud platforms such as Hugging Face Spaces or connected to cloud LLM APIs (OpenAI, Google Gemini, AWS Bedrock).

Evaluating a Generative AI application is essential to ensure output quality before and after deployment. Common automatic evaluation metrics include ROUGE and BLEU (which measure n-gram overlap between generated text and a reference, commonly used for summarization/translation) and perplexity (which measures how well a language model predicts a sample). Together with qualitative human review, these metrics allow developers to monitor model performance, detect regressions, and compare different models/prompts objectively.

This end-to-end exercise integrates model deployment with quantitative evaluation, reflecting real-world MLOps practice for Generative AI systems.

ALGORITHM

1. Define an inference function that takes user text input and returns model-generated output.
2. Wrap the inference function using the Gradio Interface API with appropriate input/output components.
3. Launch the Gradio app locally (and optionally deploy to Hugging Face Spaces for cloud hosting).
4. Prepare a small evaluation set of generated outputs and corresponding reference texts.
5. Compute ROUGE scores between generated and reference texts using the 'evaluate' library.
6. Report and interpret the evaluation metrics.

PROGRAM / CODE

```
import gradio as gr
from transformers import pipeline
import evaluate
```

```
# ----- 1. Build and Deploy the App -----
summarizer = pipeline("summarization", model="facebook/bart-large-cnn")

def summarize_text(input_text):
    result = summarizer(input_text, max_length=45, min_length=15, do_sample=False)
    return result[0]["summary_text"]

demo = gr.Interface(
    fn=summarize_text,
    inputs=gr.Textbox(lines=8, label="Enter text to summarize"),
    outputs=gr.Textbox(label="Generated Summary"),
    title="GenAI Text Summarizer",
    description="A cloud-deployable Generative AI summarization app built with Gradio."
)
demo.launch(share=True) # share=True generates a public cloud URL

# ----- 2. Evaluate Generated Output -----
rouge = evaluate.load("rouge")

generated_summaries = [
    "AI models generate new content such as text and images.",
]
reference_summaries = [
    "Generative AI models are capable of producing new content including text and images.",
]

scores = rouge.compute(predictions=generated_summaries, references=reference_summaries)
print("ROUGE Evaluation Scores:", scores)
```

SAMPLE INPUT

Long article text submitted via the Gradio web interface + one generated/reference summary pair for evaluation

SAMPLE OUTPUT

```
Running on local URL: http://127.0.0.1:7860
Running on public URL: https://xxxxx.gradio.live

ROUGE Evaluation Scores: {'rouge1': 0.78, 'rouge2': 0.55, 'rougeL': 0.74, 'rougeLsum': 0.74}
```

RESULT

A Generative AI text-summarization application was successfully deployed as a cloud-accessible web app using Gradio, and its output quality was evaluated using ROUGE metrics.